

# 单周期 CPU 改进实验报告

2026 年 5 月 26 日

## 1 实验目的

1. 深入理解单周期 CPU 的工作原理和指令执行流程
2. 实现 MIPS 指令集中的 sub（有符号减法）和 bgtz（大于零跳转）指令
3. 编写汇编程序计算斐波那契数列第 10 项，验证新增指令的正确性
4. 通过实验箱验证，观察 CPU 的执行过程和最终结果

## 2 实验原理

### 2.1 单周期 CPU 基本结构

单周期 CPU 在一个时钟周期内完成一条指令的全部执行过程，包括取指、译码、执行、访存和写回五个阶段。CPU 主要由程序计数器 PC、指令存储器、寄存器堆、ALU、数据存储器和控制单元组成。

### 2.2 新增指令格式与功能

#### 2.2.1 sub 指令（R 型）

sub 是有符号减法指令，属于 R 型指令，其指令格式如下：

| op(6 位) | rs(5 位) | rt(5 位) | rd(5 位) | sa(5 位) | funct(6 位) |
|---------|---------|---------|---------|---------|------------|
| 000000  | 源寄存器 1  | 源寄存器 2  | 目标寄存器   | 00000   | 100010     |

表 1: sub 指令格式

功能：将寄存器 rs 的值减去寄存器 rt 的值，结果存入寄存器 rd 中，即  $rd = rs - rt$ （有符号数运算）。

### 2.2.2 bgtz 指令 (I 型)

bgtz 是大于零跳转指令，属于 I 型指令，其指令格式如下：

|         |         |            |              |
|---------|---------|------------|--------------|
| op(6 位) | rs(5 位) | 00000(5 位) | offset(16 位) |
| 000111  | 源寄存器    | 保留位        | 偏移量          |

表 2: bgtz 指令格式

功能：如果寄存器 rs 的值大于 0（有符号数），则跳转到目标地址执行。目标地址计算公式为： $PC = PC + offset \times 4$ （本实验 CPU 采用此非标准公式，与原始代码保持一致）。

## 3 实现步骤

### 3.1 新增 sub 和 bgtz 指令

修改 single\_cycle\_cpu.v 文件，不改变原有指令的功能。

#### 3.1.1 添加指令识别

在”实现指令列表”部分添加新指令的 wire 定义和赋值：

```
1 // 新增：有符号减法和大于零跳转指令
2 wire inst_SUB, inst_BGTZ;
3
4 // 新增指令解码
5 assign inst_SUB = op_zero & sa_zero & (funct == 6'b100010); // 有符号
   减法
6 assign inst_BGTZ = (op == 6'b000111); // 大于零跳转
```

#### 3.1.2 添加分支跳转逻辑

在”分支跳转”部分添加 bgtz 指令的跳转条件判断：

```
1 // 新增：bgtz 跳转条件
2 wire bgtz_taken;
3 // BGTZ 跳转条件：GPR[rs] > 0（有符号数，最高位为0且非零）
4 assign bgtz_taken = (rs_value[31] == 1'b0) && (rs_value != 32'd0);
```

修改跳转信号 jbr\_taken，添加 bgtz 指令的跳转判断：

```

1 assign jbr_taken = j_taken // 指令跳转：无条件跳转 或 满足分支跳转条
   件
2         | inst_BEQ & beq_taken
3         | inst_BNE & bne_taken
4         | inst_BGTZ & bgtz_taken; // 新增：大于零跳转

```

### 3.1.3 更新 ALU 控制信号

修改 inst\_sub 赋值，将 sub 指令加入减法运算：

```

1 assign inst_sub = inst_SUBU | inst_SUB; // 减法（包含有符号和无符号）

```

### 3.1.4 更新写回逻辑

修改 inst\_wdest\_rd 赋值，将 sub 指令加入寄存器写回列表：

```

1 assign inst_wdest_rd = inst_ADDU | inst_SUBU | inst_SUB | inst_SLT |
   inst_AND | inst_NOR | inst_OR | inst_XOR | inst_SLL | inst_SRL;

```

## 3.2 实现斐波那契数列计算

修改 inst\_rom.v 文件，在原有指令之后添加斐波那契计算程序。

### 3.2.1 扩展指令存储器

将指令存储器大小从 20 条扩展到 32 条：

```

1 wire [31:0] inst_rom[31:0]; // 扩展到32条指令

```

### 3.2.2 编写斐波那契汇编程序

根据实验要求的逻辑，编写计算 fib(10) 的汇编程序，并适配原始 CPU 的跳转逻辑：

```

1 // 原有指令保持不变（0-18号，00H-48H）
2 assign inst_rom[ 0] = 32'h24010001; // 00H: addiu $1 , $0, #1
3 assign inst_rom[ 1] = 32'h00011100; // 04H: sll   $2 , $1, #4
4 // ... 中间原有指令省略 ...
5 assign inst_rom[18] = 32'h3C0C000C; // 48H: lui   $12, #12
6
7 // 斐波那契计算指令
8 assign inst_rom[19] = 32'h240D000A; // 4CH: addiu $13, $0, #10 | $13
   =10 (n=10)

```

```

9 assign inst_rom[20] = 32'h240E0002; // 50H: addiu $14,$0,#2      | $14
    =2 (idx=2)
10 assign inst_rom[21] = 32'h240F0001; // 54H: addiu $15,$0,#1      | $15
    =1 (fib(1))
11 assign inst_rom[22] = 32'h24100001; // 58H: addiu $16,$0,#1      | $16
    =1 (fib(2))
12 assign inst_rom[23] = 32'h01F08821; // 5CH: addu  $17,$15,$16    | Loop
    入口: 计算fib(n)
13 assign inst_rom[24] = 32'h25CE0001; // 60H: addiu $14,$14,#1      | idx
    ++
14 assign inst_rom[25] = 32'h00107821; // 64H: move  $15,$16          | $15
    = fib(n-1)
15 assign inst_rom[26] = 32'h00118021; // 68H: move  $16,$17          | $16
    = fib(n)
16 assign inst_rom[27] = 32'h01AE9022; // 6CH: sub   $18,$13,$14    | $18
    = n - idx (验证sub)
17 assign inst_rom[28] = 32'h1E40FFFB; // 70H: bgtz  $18,Loop        | 若
    $18>0跳回5CH(验证bgtz)
18 assign inst_rom[29] = 32'hAC110004; // 74H: sw    $17,4($0)       | Mem
    [1] = fib(10)=55
19 assign inst_rom[30] = 32'h08000000; // 78H: j     00H            | 跳转
    回开头循环执行
20 assign inst_rom[31] = 32'd0;          // 7CH: 空指令

```

### 3.2.3 更新指令读取逻辑

扩展 case 语句，支持读取 32 条指令：

```

1 always @(*)
2 begin
3     case (addr)
4         5'd0 : inst <= inst_rom[0 ];
5         // ... 中间原有case项省略 ...
6         5'd30: inst <= inst_rom[30];
7         5'd31: inst <= inst_rom[31];
8         default: inst <= 32'd0;
9     endcase
10 end

```

## 4 实验原理图

表 3: 斐波那契数列计算汇编指令详解

| 地址                  | 机器码      | 汇编指令                | 功能说明                                            |
|---------------------|----------|---------------------|-------------------------------------------------|
| 初始化阶段               |          |                     |                                                 |
| 4CH                 | 240D000A | addiu \$13,\$0,#10  | GPR[13] = 10, 存储要计算的项数 n=10                     |
| 50H                 | 240E0002 | addiu \$14,\$0,#2   | GPR[14] = 2, 循环计数器 idx 初始值                      |
| 54H                 | 240F0001 | addiu \$15,\$0,#1   | GPR[15] = 1, 存储 fib(1)                          |
| 58H                 | 24100001 | addiu \$16,\$0,#1   | GPR[16] = 1, 存储 fib(2)                          |
| 循环阶段 (Loop 入口: 5CH) |          |                     |                                                 |
| 5CH                 | 01F08821 | addu \$17,\$15,\$16 | GPR[17] = GPR[15] + GPR[16], 计算当前 fib 值         |
| 60H                 | 25CE0001 | addiu \$14,\$14,#1  | GPR[14] = GPR[14] + 1, 计数器加 1                   |
| 64H                 | 00107821 | move \$15,\$16      | GPR[15] = GPR[16], 前项后移                         |
| 68H                 | 00118021 | move \$16,\$17      | GPR[16] = GPR[17], 当前项变前项                       |
| 6CH                 | 01AE9022 | sub \$18,\$13,\$14  | GPR[18] = GPR[13] - GPR[14], 计算剩余项数 (验证 sub 指令) |
| 70H                 | 1E40FFFB | bgtz \$18,Loop      | 若 GPR[18] > 0 则跳转到 5CH (验证 bgtz 指令)             |
| 结束阶段                |          |                     |                                                 |
| 74H                 | AC110004 | sw \$17,4(\$0)      | Mem[1] = GPR[17], 将 fib(10) 存入数据存储器地址 4         |
| 78H                 | 08000000 | j 00H               | 无条件跳转到程序开头, 循环执行                                |

## 5 实验结果

### 5.1 实验箱验证结果

将程序下载到 FPGA 实验箱, 通过触摸屏观察结果:

1. 在触摸屏上输入内存地址 4 (0x4)
2. 观察 MDATA 显示的值为 55 (十六进制 0x37), 与预期一致

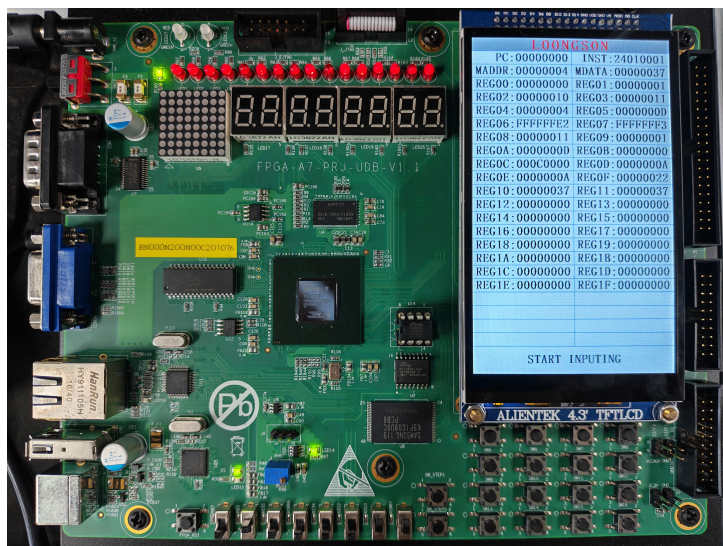


图 1: 实验箱运行结果照片

## 6 问题与解决

在实验过程中遇到了一个关键问题：最初编写的 `bgtz` 指令偏移量错误，导致程序跳转到错误的地址，使得寄存器 `$16` 一直在 1 和 2 之间跳动。

经过分析发现，原始 CPU 的分支跳转逻辑采用了非标准的计算公式  $PC = PC + offset \times 4$ ，而不是标准 MIPS 的  $PC = (PC + 4) + offset \times 4$ 。由于实验要求不能修改原有指令，因此不能修改 CPU 核心的跳转逻辑，只能调整新增指令的偏移量。

重新计算偏移量：目标地址 `0x5C`，`bgtz` 指令地址 `0x70`，因此  $offset = (0x5C - 0x70)/4 = -5$ ，对应的 16 位补码为 `0xFFFFB`。修改后程序正常运行，得到了正确的结果。